

DocAlign User Personas

Owner: James (Product)

Status: Draft for review

Date: July 2026

Source: Derived from the DocAlign 1.0 engineering handoff (Priya Anand, Engineering)

Purpose

These personas describe the people who interact with DocAlign, so that Product, Engineering, and Documentation can design and write for specific, representative users rather than a generic "user." Each persona is a decision-making tool: when a design or documentation choice arises (how much to explain, what to assume, what a finding should say), the personas provide concrete reference points to test the choice against.

The set covers one **primary user** (the technical writer who operates DocAlign), two **secondary users** (who encounter DocAlign's output but do not operate it), and two **edge-case situations** (which stress DocAlign's assumptions and shape specific parts of the product and its documentation).

Primary user

Maya Chen — Senior Technical Writer

Context. Maya is a senior technical writer at a mid-sized software company, embedded with a product engineering team. She is fluent in structured authoring and documentation tools, comfortable in a docs-as-code workflow, and works in Git daily — she can open a pull request, resolve a merge, and read a YAML file without friction. She is technical for a writer, but she is not an engineer and does not write production code. She owns documentation for several products and is accountable for its accuracy.

Goals.

- Keep documentation accurate as code changes, without manually policing every code change.
- Catch drift before it ships to users.
- Spend her time on high-value writing (concepts, structure, clarity) rather than hunting for what changed.
- Have confidence that nothing important slipped past her.

Frustrations.

- Often learns documentation is wrong only after a user complains or a support ticket surfaces.
- Developers change things without telling her, and she cannot attend every code review.
- Feels perpetually behind — reactive rather than proactive.
- The invisible labor of staying current is exhausting and under-recognized.

How she uses DocAlign. Maya is the primary operator. She configures DocAlign, monitors its findings on pull requests, and makes the Accept / Reject / Revise decisions. She runs the human-in-the-loop workflow and relies on DocAlign as a safety net, interacting with it most working days.

What she needs from documentation. Because Maya is technical and Git-comfortable, documentation can assume real fluency — it need not explain Git or CI/CD from scratch, and over-explaining wastes her time. What she needs is thorough, precise reference material: exactly what each configuration parameter does, how each finding type behaves, what each severity means, and how auto-remediation decides what it touches. The Configuration Quick Reference and the Accept/Reject/Revise Workflow Guide must be complete and exact. Her documentation should be efficient, reference-dense, and respectful of her expertise.

Secondary users

Devan Okoye — Software Developer

Context. Devan writes production code (Python and Java) on the same product team. He lives in pull requests but thinks about code, not documentation; docs are peripheral to his day. He is highly technical but has little patience for documentation processes and does not consider docs his job.

Goals.

- Ship his code without friction.
- Understand quickly whether a DocAlign finding on his PR requires anything of him, or whether it is the writer's concern.
- Avoid having a documentation tool slow down his merge or demand work he sees as outside his role.

Frustrations.

- A tool that blocks his merge for reasons he does not understand.
- Findings that are unclear about who needs to act.

- Anything that feels like documentation bureaucracy imposed on his workflow. He disengages fast if DocAlign feels like noise.

How he uses DocAlign. Devan does not configure or operate DocAlign. He sees its comment on his pull request and needs to grasp, in seconds, what it means and whether it is his problem. His entire interaction is: read the PR comment, understand it, move on (or occasionally act).

What he needs from documentation. Devan needs the opposite register from Maya. The PR comment itself must be immediately scannable and explicit about ownership — "does this block my merge? do I need to do anything, or is this the writer's job?" — answerable at a glance. He will not read a guide. His need is met primarily in the **design of the findings and severity indicators**, not in a document he is expected to read. He shapes the product's output, not a manual.

Ana Reyes — Release Manager

Context. Ana coordinates releases across multiple teams. She is moderately technical — comfortable with pipelines and process, but does not write code. She cares about what is shippable and what is blocking a release, and she sees DocAlign's findings in aggregate through that lens.

Goals.

- Know whether documentation drift is blocking a release, and if so, what must happen to unblock it.
- Have a clear signal of release-readiness.
- Avoid surprises late in the release cycle.

Frustrations.

- Discovering a merge-blocking finding at the last minute.
- Ambiguity about whether something is a hard blocker or advisory.
- Not knowing who owns resolving a finding, so she can direct it to the right person.

How she uses DocAlign. Ana monitors findings at the release level, particularly Critical findings that block merges. She is less interested in individual drift items than in the aggregate picture of what is blocking a release. She may consult the audit log to confirm findings were resolved.

What she needs from documentation. Ana needs the severity-and-blocking behavior to be crystal clear: precisely what blocks a merge, what is advisory, how blocking is configured, and how to see resolution status. Her needs point the documentation toward clarity on the severity model and merge-blocking rules, and toward making the audit log legible to a non-developer. A

concise explanation of severities aimed at someone managing process — rather than operating the tool — would serve her well.

Edge-case situations

Sam Bright — Developer Without a Writer

Context. Sam is a developer on a small team or startup with no technical writer at all. Sam writes the code and is also expected to maintain its documentation — not by inclination or training, but by necessity, because there is no one else. Sam is technical about code but a genuine novice about documentation: unfamiliar with structured authoring, with what "good docs" look like, and with the conventions a professional writer takes for granted. Sam may not distinguish a concept topic from a task topic and may not grasp why drift matters until a user is harmed by it. Sam is technically fluent but documentation-naive — the inverse of the common assumption that a technical reader is a documentation-capable reader.

Goals.

- Avoid shipping broken documentation, without becoming a documentation expert to do so.
- Have DocAlign explain what is wrong in plain terms and guide a fix without presupposing documentation knowledge.
- Do the right thing with minimal ramp-up.

Frustrations.

- Findings that assume documentation literacy Sam lacks — terms like "concept topic" or "single-sourcing," or severity language that presumes Sam knows why a "Minor" style issue matters.
- Being told there is drift without being told, in accessible terms, what to actually do about it.
- Feeling out of their depth in a domain they were drafted into unwillingly.

How they use DocAlign. Sam is both the code author who triggered the drift and the person who must respond to the finding — collapsing the writer's and developer's usually separate roles into one overwhelmed person. Sam configures DocAlign minimally and acts on findings, but without a professional writer's fluency.

What this means for the product and documentation. Sam breaks the assumption that a technical reader is a documentation-capable reader. Documentation written only for Maya would leave Sam lost. This edge case argues for an **accessible layer** in the documentation: plain-language explanations of why drift matters, findings that do not presume knowledge of documentation conventions, and guidance that teaches a little as it directs. Concretely, it argues

for the Getting Started Guide to have an entry point that assumes code fluency but not documentation fluency, and for findings to explain themselves in terms a developer with no documentation background can act on.

Sam is also the user most likely to over-trust DocAlign — for example, revising documentation to match buggy code because they lack the judgment to push back. The human-in-the-loop workflow and the explainability of findings matter most for Sam, who is the least-equipped human in the loop. (See the related open question on whether DocAlign should assume code is always the source of truth.)

The Enterprise Documentation Team

Context. A large enterprise where documentation and engineering are separate organizations — a docs team distinct from the dev teams, possibly in different reporting lines, working across many repositories, including monorepos that contain multiple products. Findings must reach the correct person across an organizational boundary, and analysis must be scoped correctly so that one product's findings do not appear on another product's pull requests.

Goals.

- Route findings to the correct writer.
- Scope analysis correctly per product, without cross-product noise.
- Have DocAlign respect organizational structure.

Frustrations.

- Findings landing on the wrong person.
- Cross-product noise in a monorepo.
- A tool that assumes the small-team model where one person owns both code and documentation.

What this means for the product and documentation. This situation points toward the configuration documentation — specifically `notification_rules`, `excluded_paths`, and scoping/routing configuration — needing to be thorough and to include realistic multi-team, multi-product examples. The Configuration Quick Reference must serve not only the single-repository case but complex organizational setups with worked examples.

How the personas shape the documentation suite

Taken together, the personas provide the rationale for the shape of DocAlign's documentation:

- **Maya and the Enterprise team** justify a thorough, precise Configuration Quick Reference, including complex multi-team examples.

- **Sam** justifies an accessible Getting Started on-ramp that assumes code fluency but not documentation fluency, and findings that explain themselves.
- **Ana** justifies crisp, process-oriented documentation of the severity model and merge-blocking rules.
- **Devan** justifies self-explanatory findings and severity indicators at the point of contact (the PR comment), rather than a guide he will not read.
- **Maya** justifies reference-dense, efficient documentation that assumes professional fluency.

A recurring theme across the personas is that "make it clear" means different things for different readers. Maya wants completeness; Devan wants scannability; Ana wants blocking clarity; Sam wants accessibility without jargon. A single document cannot serve all of them identically, which is why the documentation suite is deliberately differentiated by audience and purpose.